

FULL STACK DEVELOPMENT

FASE 5 | AULA 05 -

CONCEITOS BÁSICOS DE DOCKER

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMSE

O QUE VEM POR AÍ?

Boas-vindas, alunos e alunas.

Nesta aula, você aprenderá o que são containers, qual sua função e como eles ajudarão a resolver problemas com recursos do computador, como CPU, memória RAM, disco etc.

Você também aprenderá como instalar o subsistema Linux no Windows, o WSL, e suas versões e distribuições (Ubuntu, Debian etc.), além de como instalar corretamente o Docker, o que é Docker, como ele funciona e quais vantagens teremos em usá-lo. Por fim, executaremos nosso primeiro container.

Bons estudos!

HANDS ON

Nesta primeira aula, faremos a instalação do WSL via Powershell no Windows; depois, com o ambiente Linux rodando, faremos a instalação do Docker Desktop e a configuração para que a integração com o WSL seja feita. É no ambiente Linux que executaremos os comandos relacionados ao Docker.

Por fim, executaremos alguns comandos básicos de Docker para entender um pouco sobre sua estrutura e como ela funciona, além de executar nosso primeiro container.

EXEMPLO

SAIBA MAIS

O Docker foi lançado em 2013 pela empresa dotCloud, e é uma ferramenta Open Source que foi criada com o intuito de melhorar o isolamento e o uso de recursos, além de rodar aplicações sem a necessidade de uma interface gráfica se comparado com uma VM (Virtual Machine = Máquina Virtual). Sua existência só foi possível graças à possibilidade de isolar processos e a virtualização presentes no kernel do Linux.

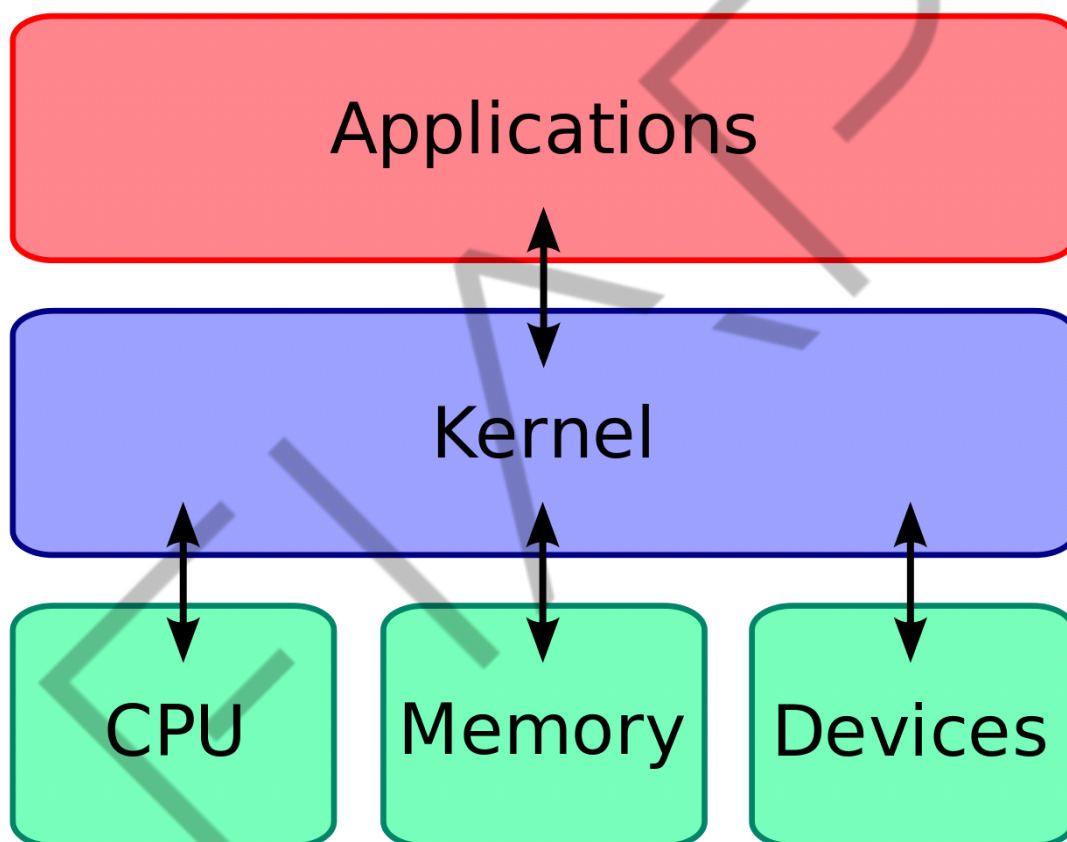


Figura 1 - Kernel Linux
Fonte: [Josh](#) (2015)

Por sua vez, o Kernel Linux é o principal componente de um sistema operacional Linux: a interface central se comunica entre o hardware e seus processos, controlando todas as principais funções e sendo responsável por gerenciar os recursos com eficiência. Em tradução livre, kernel significa núcleo.

As principais funções do kernel incluem gerenciar memória, processos, drives, chamadas do sistema e segurança. Junto do kernel temos duas outras tecnologias

que fazem com que o uso dos containers seja isolado um dos outros que se chamam Cgroups e Namespaces.

Os Cgroups são responsáveis por permitir definir limites no uso de recursos em processos Linux. Com eles, podemos estabelecer o quanto de CPU um determinado processo pode usar, priorizando os que forem mais importantes ou definidos no Cgroups. Também é possível colocar limites no uso dos recursos como CPU e memória, por exemplo, mantendo o controle de quais processos podem/devem ser reiniciados, interrompidos ou congelados com um único comando. Isso serve para controlar processos em execução em um container e esta é uma tecnologia essencial para o funcionamento correto e sem gargalos de um container.

Já Namespaces é uma tecnologia que permite criar e lidar com diferentes cenários em um mesmo sistema, permitindo analisar propriedades globais diferentes e isoladas em cada contexto. O container tem um ambiente isolado similar ao de uma VM, mas pode ter containers diferentes com aplicações diferentes rodando em paralelo em um mesmo servidor. Os Namespaces são responsáveis por criar esses isolamentos de forma transparente e lógica, sendo que cada container pode ter um conjunto de usuários, rede diferente, disco e assim por diante.

Dessa forma, caso algum dos containers que esteja rodando quebre por algum motivo, como por exemplo usar 100% de CPU, os demais containers rodando em paralelo não terão o mesmo problema de CPU. Isso garante que aplicações diferentes e que não se conversam não sejam afetadas por conta do container "ruim".

Imagine um cargueiro carregando vários containers: caso algum deles venha cair no mar, os demais não serão afetados, nem mesmo o cargueiro que representa o host em que os containers estão rodando. A figura 2 retrata exatamente como eles são.



Figura 2 - Containers
Fonte: Google Imagens (2024)

A diferença entre VMs e containers são poucas, mas mudam completamente o modo de execução e isolamento das aplicações. As figuras 3 e 4 retratam um pouco esse cenário.

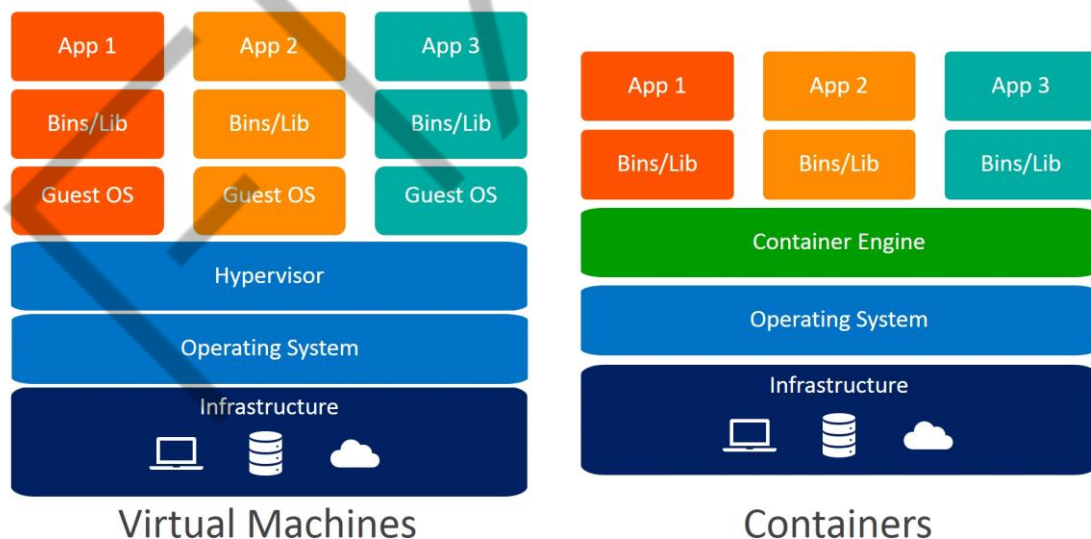


Figura 3 - Diferenças entre VMs e Containers
Fonte: Elaborado pelo autor (2024)

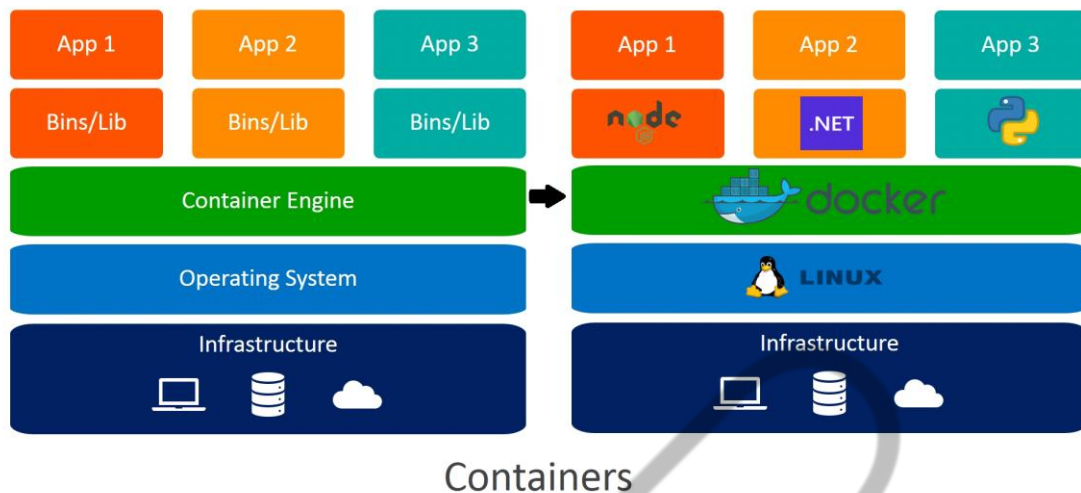


Figura 4 - Ilustrando o conceito de containers
Fonte: Elaborado pelo autor (2024)

O Docker tem algumas funcionalidades vitais para criar e rodar um container e a primeira são as imagens. As imagens são responsáveis por empacotar o sistema operacional, dependências, portas expostas, variáveis de ambiente... Existem milhares de imagens personalizadas já criadas, além de ser possível criar sua própria imagem e compartilhar com a comunidade.

Imagine que para rodar uma aplicação .NET você precise instalar a versão do .NET compatível com seu código, como instalar as dependências pelo NuGet por exemplo, aquela configuração padrão que toda aplicação .NET precisa ter pré-instalada para poder funcionar e rodar o código criado. As imagens personalizadas têm esse papel na comunidade.

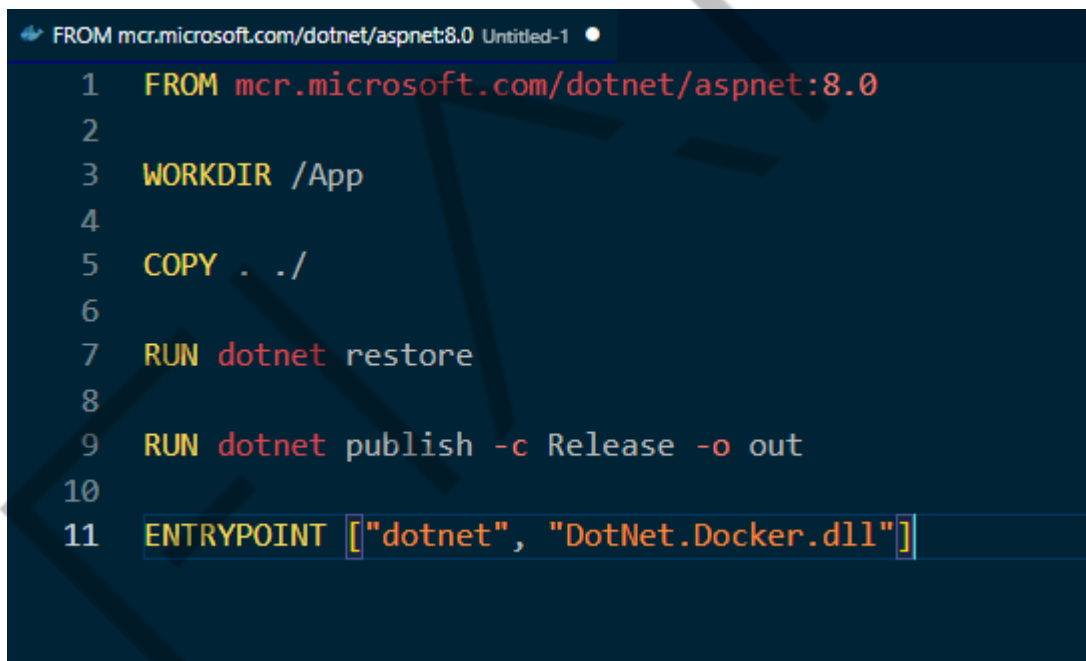
O Docker Hub é basicamente um Github voltado para imagens Docker; assim como o Github tem repositórios públicos que qualquer pessoa consegue clonar, fazer o fork e usar de fato o código que outra pessoa criou, o Docker Hub funciona de maneira bem similar. Nele encontraremos imagens oficiais e imagens não oficiais (criadas pela comunidade), seja para ajudar outras pessoas ou até mesmo para benefício próprio como meio de salvar as imagens em um local seguro (privado). Podemos utilizar essas imagens fazendo o Pull por meio da URL e tag delas.

O que são as tags?

As tags são utilizadas para definir uma versão da imagem criada, como forma de controle, atualização e organização, e fazem melhorias constantes nas imagens, seja para corrigir bugs, melhorar o processo ou até mesmo correções de segurança. É bem parecido com o Github, não é mesmo?

Para que essas imagens sejam criadas de forma personalizada, é preciso declarar os comandos, instalações, variáveis, portas etc. Isso é feito por meio de um arquivo chamado Dockerfile (o nome do arquivo já é sua extensão) e é nele que as instruções devem ser feitas. Veremos mais sobre isso na próxima aula. Assim, as figuras 5 e 6 ilustram um Dockerfile básico e outro avançado.

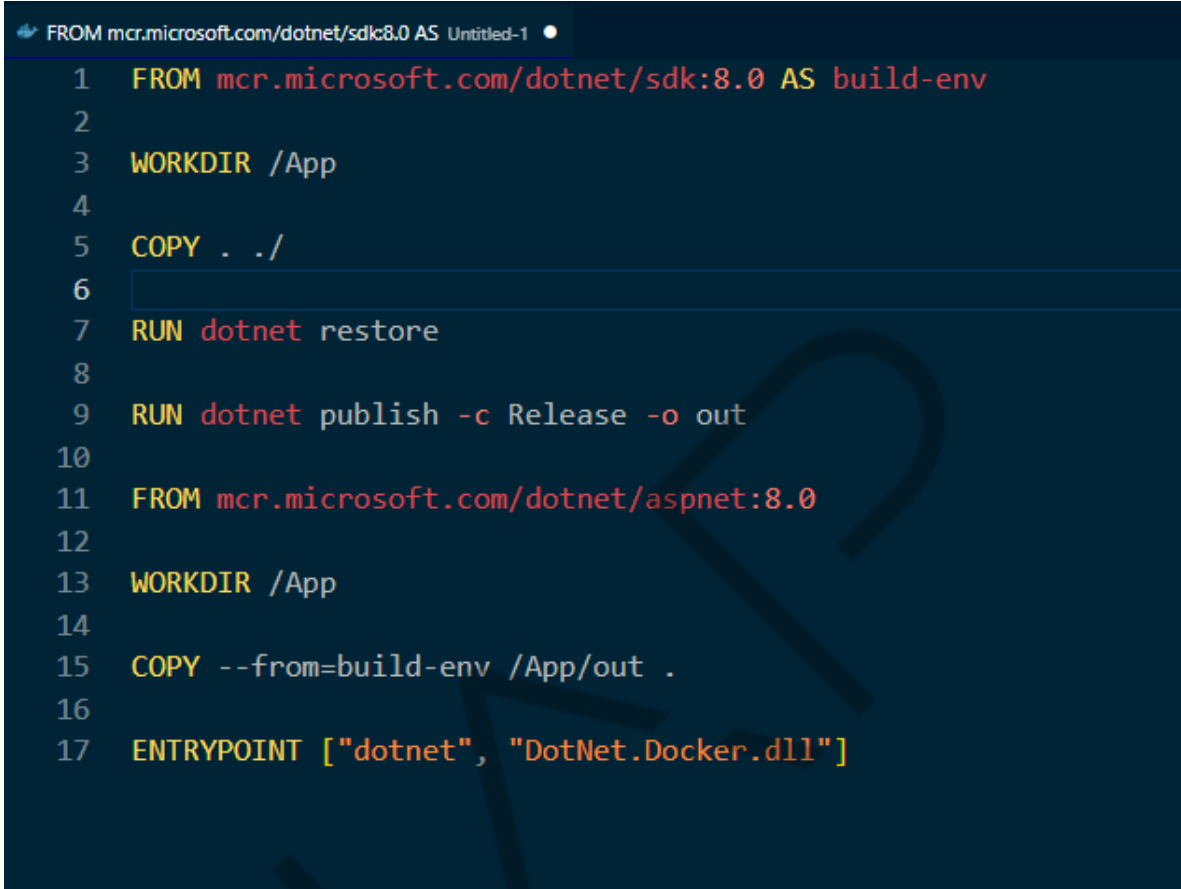
Dockerfile simples



```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 Untitled-1
1  FROM mcr.microsoft.com/dotnet/aspnet:8.0
2
3  WORKDIR /App
4
5  COPY . ./
6
7  RUN dotnet restore
8
9  RUN dotnet publish -c Release -o out
10
11 ENTRYPOINT ["dotnet", "DotNet.Docker.dll"]
```

Figura 5 - Dockerfile simples
Fonte: Elaborado pelo autor (2024)

Dockerfile multistage



```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build-env
WORKDIR /App
COPY . ./
RUN dotnet restore
RUN dotnet publish -c Release -o out
FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /App
COPY --from=build-env /App/out .
ENTRYPOINT ["dotnet", "DotNet.Docker.dll"]
```

Figura 6 - Dockerfile multistage
Fonte: Elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você viu como os containers funcionam, a distribuição Linux e a possibilidade de usá-la no Windows por meio do WSL. Também aprendeu sobre o Docker e seus principais comandos.

Além disso, você criou o primeiro container e entendeu como ele funciona e qual seu papel. O básico sobre esses temas está disponível em texto, vídeo e imagens: aproveitem o conteúdo, pois as próximas aulas serão mais hands on do que essa.

REFERÊNCIAS

JOSH. **Demystifying The Linux Kernel**. 2015. Disponível em: <<https://diligent.com/blog/demystifying-the-linux-kernel/>>. Acesso em: 29 abr. 2024.

MICROSOFT BUILD. **Como instalar o Linux no Windows com o WSL**. [learn.microsoft.com.br](https://learn.microsoft.com.br/windows/wsl/install). 2023. Disponível em: <<https://learn.microsoft.com/pt-br/windows/wsl/install>>. Acesso em: 29 abr. 2024.

RED HAT. **O que é o kernel Linux**. 2019. Disponível em: <<https://www.redhat.com/pt-br/topics/linux/what-is-the-linux-kernel>>. Acesso em: 29 abr. 2024.

ROGÉRIO, P. **Containers: o que são namespaces e cgroups**. etcd.dev. 2021. Disponível em: <<https://etcd.dev/2021/09/20/containers-o-que-sao-namespaces-e-cgroups/>>. Acesso em: 29 abr. 2024.

PALAVRAS-CHAVE

Palavras-chave: Docker. Container. Linux. WSL.

EMSE

POSTECH